

Practical-Week-12-2520080033

Task-1

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
        usleep(100000);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
        usleep(150000);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    printf("\nProducer-Consumer completed successfully.\n");

    return 0;
}
```

```
root@Shamikahasini:~# nano producer_consumer.c
root@Shamikahasini:~# nano producer_consumer1.c
root@Shamikahasini:~# gcc producer_consumer1.c -o producer_consumer1 -pthread
root@Shamikahasini:~# ./producer_consumer1
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Consumed: 8
Consumed: 9
Consumed: 10

Producer-Consumer completed successfully.
root@Shamikahasini:~# |
```

Observation

The Producer-Consumer problem was successfully implemented using **POSIX threads, counting semaphores, and mutex locks**. The semaphores synchronized access to the shared buffer, preventing overflow and underflow. Different buffer sizes were tested, and larger buffers generally reduced waiting and improved throughput.

Task-2

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1;
pthread_mutex_t resource2;

void *thread1(void *arg)
{
    printf("Thread 1: Locking Resource 1\n");
    pthread_mutex_lock(&resource1);

    sleep(1);

    printf("Thread 1: Waiting for Resource 2\n");
    pthread_mutex_lock(&resource2);

    printf("Thread 1: Acquired both resources\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    printf("Thread 2: Locking Resource 2\n");
    pthread_mutex_lock(&resource2);

    sleep(1);

    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);
}
```

```
    printf("Thread 2: Waiting for Resource 1\n");
    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired both resources\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}
```

```
root@Shamikahasini:~# nano deadlock.c
root@Shamikahasini:~# gcc deadlock.c -o deadlock -pthread
root@Shamikahasini:~# ./deadlock
Thread 2: Locking Resource 2
Thread 1: Locking Resource 1
Thread 2: Waiting for Resource 1
Thread 1: Waiting for Resource 2
^C
root@Shamikahasini:~# |
```

Observation

The two threads acquired different resources and then waited for each other's resources, causing a **deadlock due to circular waiting**. The program remained blocked, demonstrating the occurrence of deadlock.